

Lab-1

Program to simulate contiguous memory allocation techniques: Worst Fit ,Best Fit, First Fit.

Introduction

Contiguous memory allocation is a memory management technique where a process is allocated a single, continuous block of memory. This approach is simple and efficient, as it allows for easy memory allocation and deallocation. In contiguous memory allocation, each process is given a contiguous block of memory, which can be allocated using the First-Fit, Best-Fit, or Worst-Fit algorithms.

Algorithm

Step 1. Read the number of blocks (numBlocks) and the number of files (numFiles).

Step 2. Read the sizes of the blocks into an array (block[]).

Step 3. Read the sizes of the files into an array (file[]).

Step 4. Initialize: Create an array (blockFlag[]) to track allocated blocks, initialized to 0 (all blocks are free).

Step 5. Allocate Files:

For each file and For block:

If the block is free and can accommodate the file:

Allocate the block to the file.

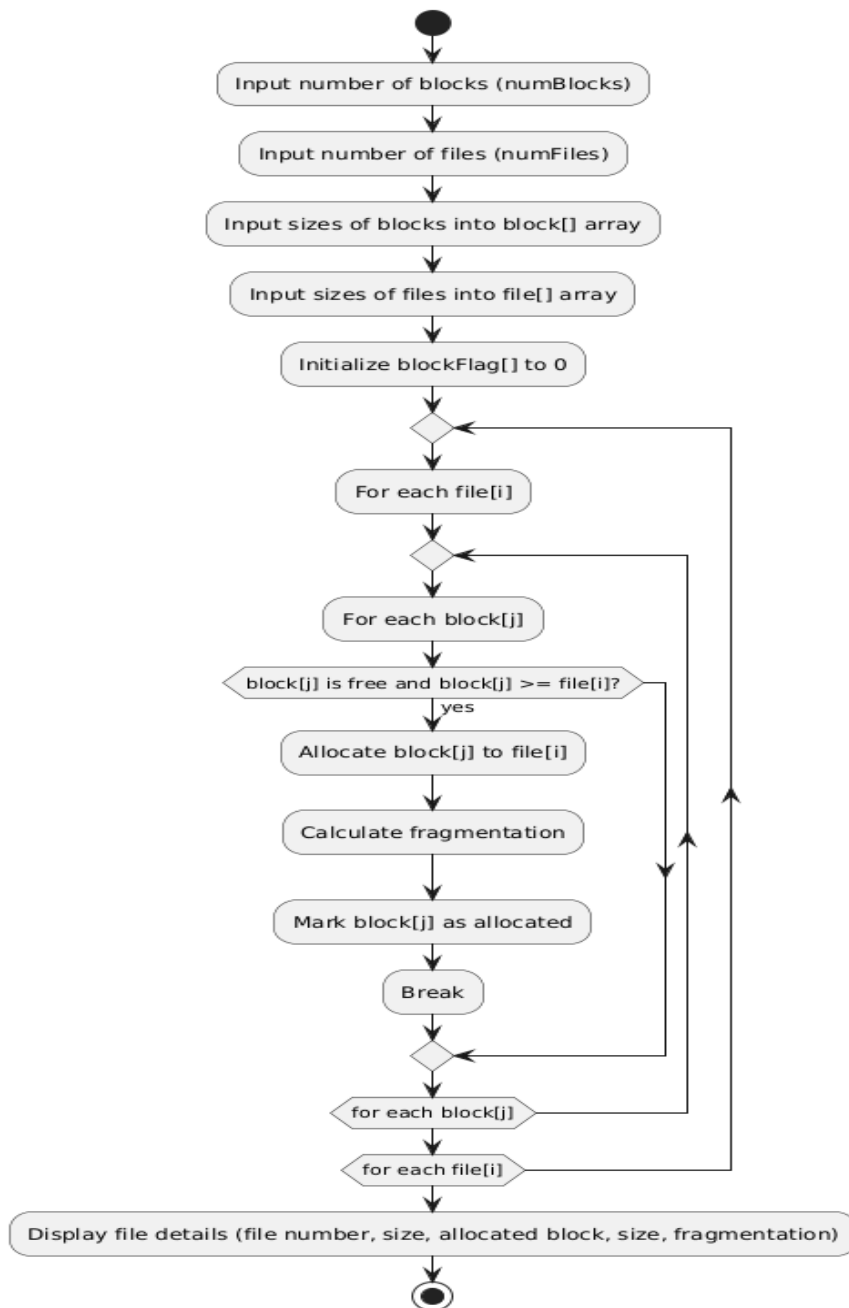
Calculate and store the fragmentation.

Mark the block as allocated.

Move to the next file.

Step 6. Print the file number, file size, allocated block number, block size, and fragmentation for each file.

Flowchart



Program Code

```
#include <stdio.h>
#include <conio.h>
#define MAX 25
int main() {
    int frag[MAX], block[MAX], file[MAX], i, j, numBlocks, numFiles, temp;
    static int blockFlag[MAX], fileBlock[MAX];
    printf("\n\tMemory Management Scheme - First Fit");
    printf("\nEnter the number of blocks: ");
    scanf("%d", &numBlocks);
    printf("Enter the number of files: ");
    scanf("%d", &numFiles);
    printf("\nEnter the size of the blocks:\n");
    for (i = 1; i <= numBlocks; i++) {
        printf("Block %d: ", i);
        scanf("%d", &block[i]); }
    printf("Enter the size of the files:\n");
    for (i = 1; i <= numFiles; i++) {
        printf("File %d: ", i);
        scanf("%d", &file[i]);
    }
    for (i = 1; i <= numBlocks; i++) {
        blockFlag[i] = 0; }
    for (i = 1; i <= numFiles; i++) {
        for (j = 1; j <= numBlocks; j++) {
            if (blockFlag[j] != 1) {
                temp = block[j] - file[i];
                if (temp >= 0) {
                    fileBlock[i] = j;
                    frag[i] = temp;
                    blockFlag[j] = 1;
                    break; } }
        } }
    printf("\nFile No:\tFile Size:\tBlock No:\tBlock Size:\tFragmentation");
    for (i = 1; i <= numFiles; i++) {
        printf("\n%d\t%d\t%d\t%d\t%d",
            i, file[i], fileBlock[i], block[fileBlock[i]], frag[i]); }
    getch();
}
```

Output

```
Memory Management Scheme - First Fit
Enter the number of blocks: 5
Enter the number of files: 4

Enter the size of the blocks:
Block 1: 3
Block 2: 4
Block 3: 2
Block 4: 1
Block 5: 6
Enter the size of the files:
File 1: 4
File 2: 3
File 3: 4
File 4: 2

File No:      File Size:      Block No:      Block Size:      Fragmentation
1             4             2             4             0
2             3             1             3             0
3             4             5             6             2
4             2             3             2             0
```

Best-FIT

Algorithm

Step 1. Read the number of blocks (numBlocks) and the number of files (numFiles).

Step 2. Read the sizes of the blocks into an array (block[]).

Step 3. Read the sizes of the files into an array (file[]).

Step 4. Create an array (blockFlag[]) to track allocated blocks, initialized to 0 (all blocks are free).

Step 5. Allocate Files:

For each file:

Set lowest to a large value (e.g., 10000).

For each block:

If the block is free and can accommodate the file:

Calculate the leftover space (temp).

If temp is less than lowest, update lowest and record the block index.

If a suitable block is found, allocate it to the file and mark it as allocated.

Step 6. Print the file number, file size, allocated block number, block size, and fragmentation for each file.

Program Code

```
#include <stdio.h>
#include <conio.h>
#define MAX 25

int main() {
    int frag[MAX], block[MAX], file[MAX], i, j, numBlocks, numFiles, temp, lowest;
    static int blockFlag[MAX], fileBlock[MAX];
    printf("\n\tMemory Management Scheme - Best Fit");
    printf("\nEnter the number of blocks: ");
    scanf("%d", &numBlocks);
    printf("Enter the number of files: ");
    scanf("%d", &numFiles);
    printf("\nEnter the size of the blocks:\n");
    for (i = 1; i <= numBlocks; i++) {
        printf("Block %d: ", i);
        scanf("%d", &block[i]); }
    printf("Enter the size of the files:\n");
    for (i = 1; i <= numFiles; i++) {
        printf("File %d: ", i);
        scanf("%d", &file[i]); }
    for (i = 1; i <= numBlocks; i++) {
        blockFlag[i] = 0;
    }
    for (i = 1; i <= numFiles; i++) {
        lowest = 10000;
        for (j = 1; j <= numBlocks; j++) {
            if (blockFlag[j] != 1) {
                temp = block[j] - file[i];
                if (temp >= 0 && lowest > temp) {
                    fileBlock[i] = j;
                    lowest = temp; } } }
        frag[i] = lowest;
        if (fileBlock[i] != 0) {
            blockFlag[fileBlock[i]] = 1; } }
    printf("\nFile No\tFile Size\tBlock No\tBlock Size\tFragmentation");
    for (i = 1; i <= numFiles; i++) {
        if (fileBlock[i] != 0) {
            printf("\n%d\t%d\t%d\t%d\t%d",
                i, file[i], fileBlock[i], block[fileBlock[i]], frag[i]); } }
    getch(); }
```

Output

```
Memory Management Scheme - Best Fit
Enter the number of blocks: 5
Enter the number of files: 4

Enter the size of the blocks:
Block 1: 2
Block 2: 4
Block 3: 3
Block 4: 1
Block 5: 5
Enter the size of the files:
File 1: 6
File 2: 3
File 3: 2
File 4: 5

File No File Size      Block No      Block Size      Fragmentation
2         3           3           3              0
3         2           1           2              0
4         5           5           5              0_
```

Worst-Fit

Algorithm

Step 1. Read nb (number of blocks) and nf (number of files).

Step 2. Read sizes of blocks into array b[] and sizes of files into array f[].

Step 3. Initialize:

Create arrays bf[] (block allocation status) and ff[] (file allocation status) initialized to 0 and -1, respectively.

Step 4. Allocate Memory:

For each file i:

Set highest to -1.

For each block j:

If block j is free and can fit file i:

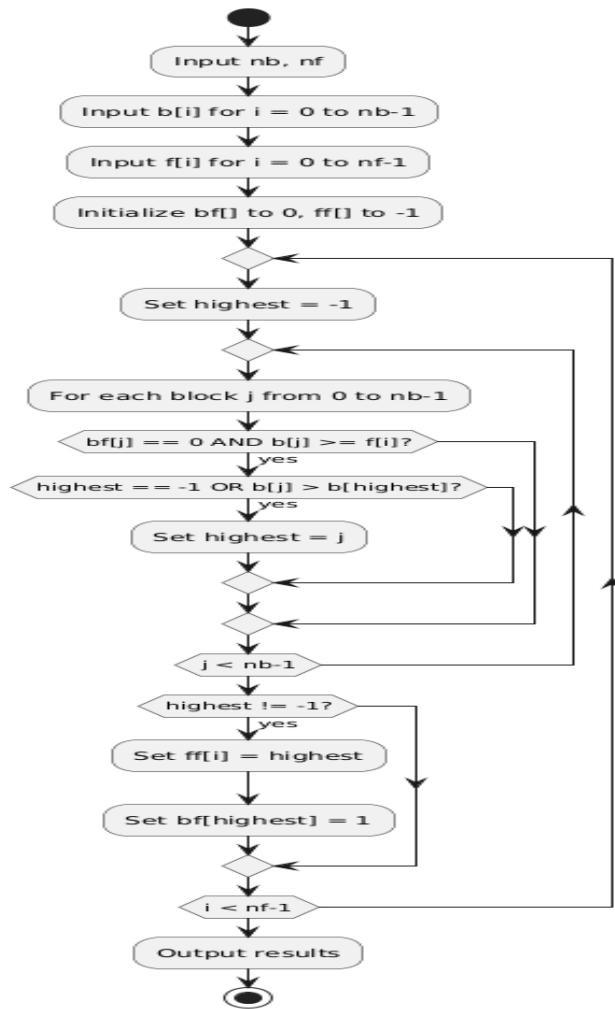
Update highest if block j is larger than the current largest suitable block.

If highest is found:

Allocate block to file, mark it as used, and calculate fragmentation.

Step 5. Print file number, file size, allocated block number, block size, and fragmentation.

Flowchart



Program Code

```
#include <stdio.h>
#include <conio.h>
#define MAX 25
int main() {
    int frag[MAX], b[MAX], f[MAX], i, j, nb, nf, temp, highest = 0;
    static int bf[MAX], ff[MAX];
    printf("\n\tMemory Management Scheme - Worst Fit");
    printf("\nEnter the number of blocks: ");
    scanf("%d", &nb);
    printf("Enter the number of files: ");
    scanf("%d", &nf);
    printf("\nEnter the size of the blocks:\n");
    for (i = 1; i <= nb; i++) {
        printf("Block %d: ", i);
        scanf("%d", &b[i]);
    }
    printf("Enter the size of the files:\n");
    for (i = 1; i <= nf; i++) {
        printf("File %d: ", i);
        scanf("%d", &f[i]); }
    for (i = 1; i <= nb; i++) {
        bf[i] = 0; }
    for (i = 1; i <= nf; i++) {
        highest = -1;
        for (j = 1; j <= nb; j++) {
            if (bf[j] != 1) {
                temp = b[j] - f[i];
                if (temp >= 0 && (highest == -1 || b[highest] < b[j])) {
                    highest = j; } } }
        if (highest != -1) {
            ff[i] = highest;
            bf[highest] = 1;
            frag[i] = b[highest] - f[i];
        } else {
```



```

        ff[i] = -1;
        frag[i] = -1; } }
printf("\nFile_no:\tFile_size:\tBlock_no:\tBlock_size:\tFragmentation");
for (i = 1; i <= nf; i++) {
    printf("\n%d\t\t%d\t\t%d\t\t%d\t\t%d", i, f[i], ff[i], (ff[i] != -1 ? b[ff[i]] : 0),
(ff[i] != -1 ? frag[i] : 0)); }
    getch();}

```

Output

```

Memory Management Scheme - Worst Fit
Enter the number of blocks: 4
Enter the number of files: 3

Enter the size of the blocks:
Block 1: 1
Block 2: 2
Block 3: 3
Block 4: 4
Enter the size of the files:
File 1: 3
File 2: 2
File 3: 1

File_no:      File_size:      Block_no:      Block_size:      Fragmentation
1             3             4             4             1
2             2             3             3             1
3             1             2             2             1
-----
Process exited after 15.57 seconds with return value 0

```

Lab-2

Write a Program to create multi-threaded process.

Introduction

A multithreaded process in an operating system is a process that can create multiple threads of execution within itself. Each thread runs independently but shares the same resources and memory space of the process, allowing for concurrent execution of tasks. This enhances performance and responsiveness, as threads can be scheduled to run simultaneously, making better use of CPU resources.

Algorithm

Step 1. **Initialize:** Declare a global variable g and a mutex lock and define a thread function myThreadFun.

Step2.Thread Function:

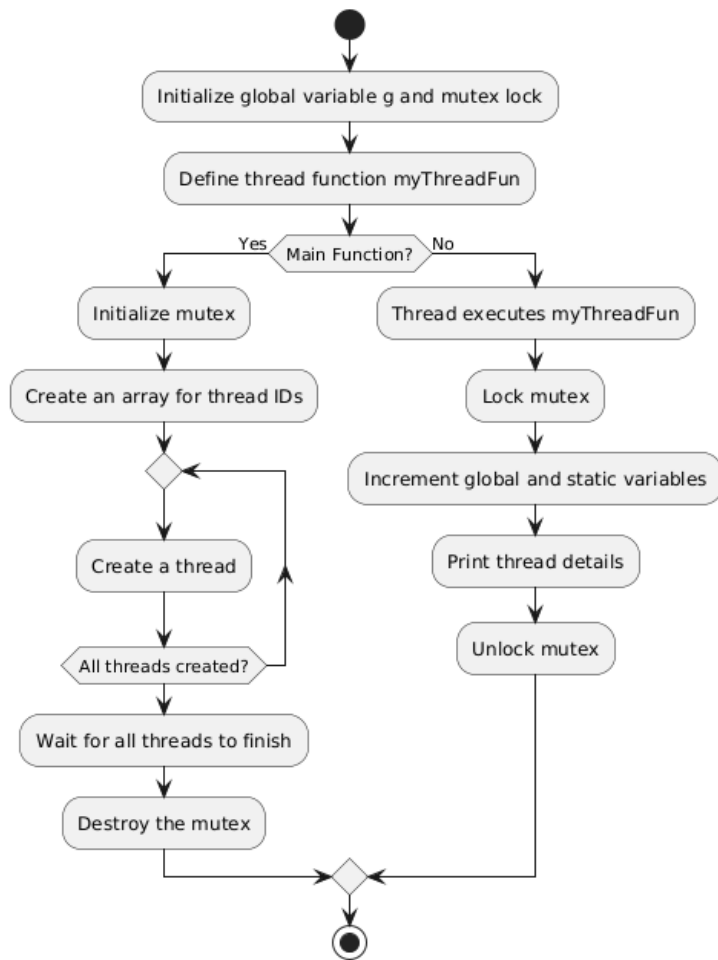
- Lock the mutex.
- Increment shared variables (g and static s).
- Print thread ID, static, and global values.
- Unlock the mutex.

Step3.Main Function: Initialize the mutex.

- Create threads, passing unique IDs.
- Wait for threads to finish with pthread_join.
- Destroy the mutex.

Step4.Exit the program.

Flowchart



Program code

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <pthread.h>

int g = 0;

void *myThreadFun(void *vargp) {

    int *myid = (int *)vargp;
```

```
static int s = 0; ++s;

++g;

printf("Thread ID: %d, Static: %d, Global: %d\n", *myid, ++s, ++g);
}

int main() {

    int i;

    pthread_t tid;

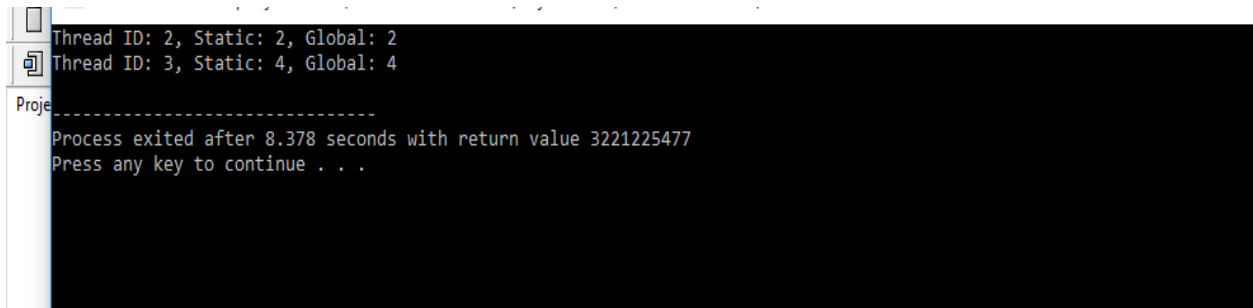
    for (i = 0; i < 3; i++) {

        pthread_create(&tid, NULL, myThreadFun, (void *)&tid); }

    pthread_exit(NULL);

}
```

Output



```
Thread ID: 2, Static: 2, Global: 2
Thread ID: 3, Static: 4, Global: 4
Process exited after 8.378 seconds with return value 3221225477
Press any key to continue . . .
```

Lab-3

Program to simulate producer consumer problem using semaphore

Introduction

The Producer-Consumer Problem is a synchronization problem in operating systems and concurrent programming. It involves two types of processes: producers and consumers, which share a common, finite-size buffer (or queue). The Producer-Consumer problem can be effectively solved using semaphores, which are synchronization primitives that help manage access to shared resources in concurrent programming.

Algorithm

Step1. **Initialize variables:** mutex = 1 (to control access to the buffer), full = 0 (to track the number of full slots), empty = 3 (to track the number of empty slots), x = 0 (to track the number of items in the buffer)

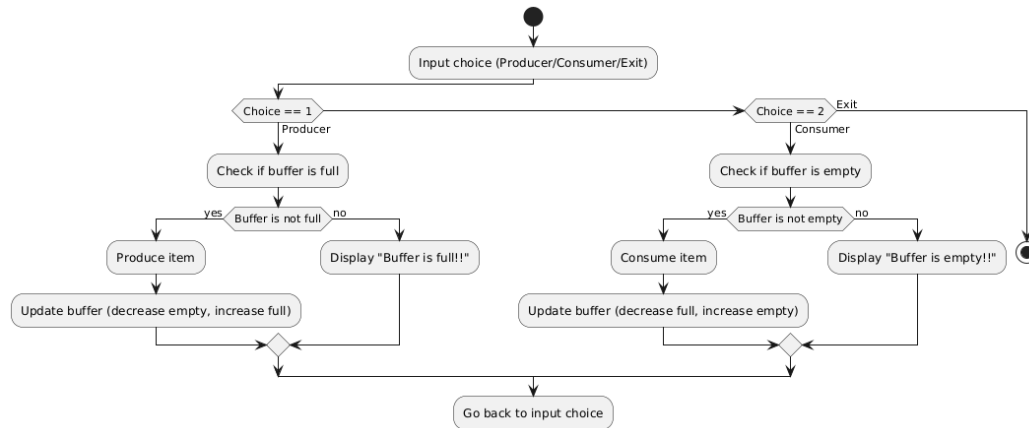
Step2. Show the options: 1.Producer,2.Consumer,3.Exit

Step 3.**Repeat until exit:**

- **If choice is 1 (Producer):**
 - If mutex == 1 and empty != 0, execute the producer:
 - Decrease empty (an item is added to the buffer).
 - Increase full (one slot is now full).
 - Increment x (produce an item).
 - Print the produced item.
 - Else, print "Buffer is full!!".
- **If choice is 2 (Consumer):**
 - If mutex == 1 and full != 0, execute the consumer:
 - Decrease full (an item is consumed from the buffer).
 - Increase empty (one slot is now empty).
 - Decrement x (consume an item).
 - Print the consumed item.
 - Else, print "Buffer is empty!!".
- **If choice is 3:**
 - Exit the program.

Step 4.**End** when user selects "Exit".

Flowchart:



Source Code

```
#include<stdio.h>
#include<stdlib.h>
int mutex=1,full=0,empty=3,x=0;
int main() {
    int n;
    void producer();
    void consumer();
    int wait(int);
    int signal(int);
    printf("\n1.Producer\n2.Consumer\n3.Exit");
    while(1){
        printf("\nEnter your choice:");
        scanf("%d",&n);
        switch(n){
            case 1: if((mutex==1)&&(empty!=0))
                producer();
            else
                printf("Buffer is full!!");
                break;
            case 2: if((mutex==1)&&(full!=0))
                consumer();
            else
                printf("Buffer is empty!!");
                break;
            case 3:
                exit(0);
                break;
        }
    }
    return 0;}
int wait(int s) {
    return (--s); }
int signal(int s) {
    return(++s); }
```

```

void producer() {
mutex=wait(mutex);
full=signal(full);
empty=wait(empty);
x++;
printf("\nProducer produces the item %d",x);
mutex=signal(mutex);    }
void consumer() {
mutex=wait(mutex);
full=wait(full);
empty=signal(empty);
printf("\nConsumer consumes item %d",x);
x--;
mutex=signal(mutex);
}

```

Output

```

1.Producer
2.Consumer
3.Exit
Enter your choice:1

Producer produces the item 1
Enter your choice:1

Producer produces the item 2
Enter your choice:2

Consumer consumes item 2
Enter your choice:2

Consumer consumes item 1
Enter your choice:1

Producer produces the item 1
Enter your choice:2

Consumer consumes item 1
Enter your choice:2
Buffer is empty!!
Enter your choice:3

-----
Process exited after 22.11 seconds with return value 0

```

Lab-4

Program to simulate and average TAT and WAITING TIME for preemptive and non-preemptive scheduling algorithm: FCFS,SJF,PRIORITY AND ROUND ROBIN.

1.First-Come First-Serve(FCFS) process scheduling algorithm without arrival time (non-preemptive)

Introduction

The FCFS is a non-preemptive scheduling algorithm in which a process is automatically queued and processing occurs according to an incoming request or process order implementation of the FCFS policy is managed with a FIFO queue. When a process enters the ready queue, its PCB is linked onto the tail of the queue. When the CPU is free, it is allocated to the process at the head of the queue.

Algorithm

Step 1: Input the processes along with their burst time(bt).

Step 2: Find waiting time (wt) for all processes.

Step 3: At first process that comes need not to wait so waiting time for process 1 will be 0 i.e. $wt[0]=0$.

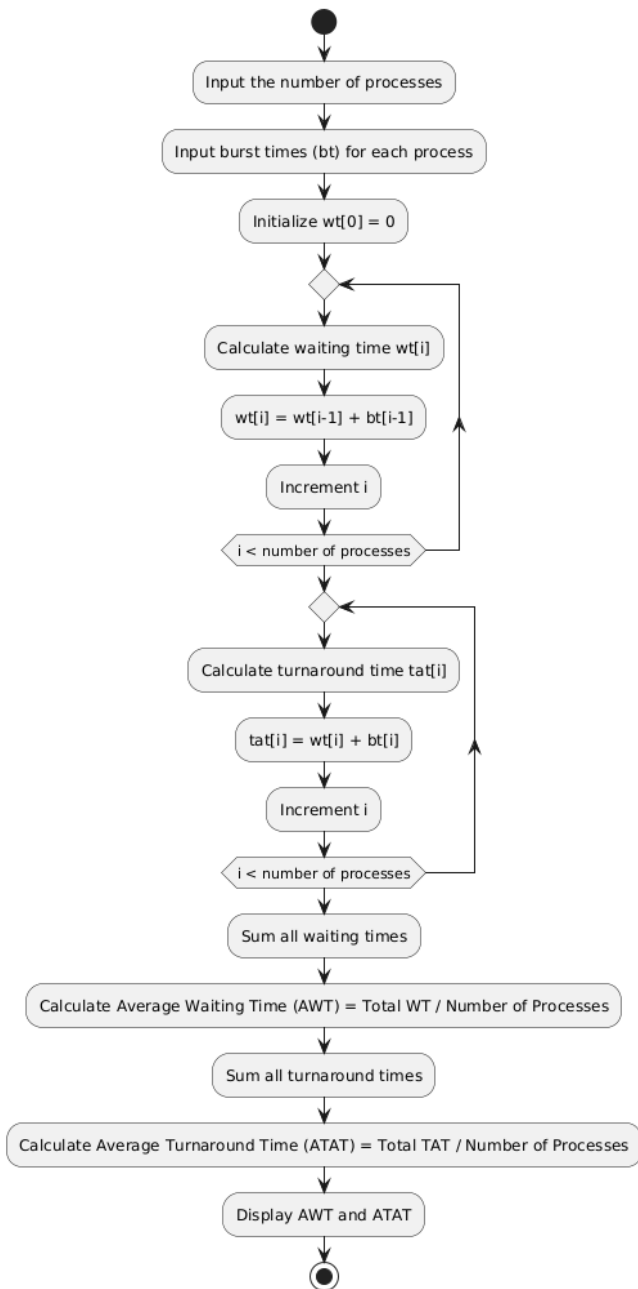
Step 4: Find waiting time for all other processes i.e. for process I will be $wt[i]= wt[i-1]+bt[i-1]$.

Step 5: Find turnaround time= waiting time + burst time for all processes.

Step 6: Find average waiting time = total waiting time/ no of process.

Step 7: Similarly, find average turnaround time = total turnaround time/no of process.

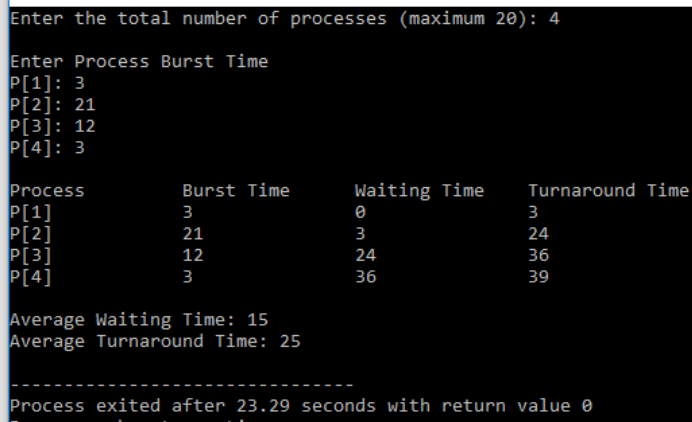
Flowchart :



Program code

```
#include<stdio.h>
int main(){
int n,bt[20],wt[20],tat[20],avwt=0,avtat=0,i,j;
printf("Enter the total number of processes(maximum
20):"); scanf("%d",&n);
printf("\nEnter Process Burst Time\n");
for(i=0;i<n;i++){
printf("P[%d]:",i+1);
scanf("%d",&bt[i]);
}
wt[0]=0;
for(i=1;i<n;i++) {
wt[i]=0;
for(j=0;j<i;j++)
wt[i]+=bt[j]; }
printf("\nProcess\t\tBurst Time\tWaiting Time\tTurnaround Time");
for(i=0;i<n;i++) {
tat[i]=bt[i]+wt[i];
avwt+=wt[i];
avtat+=tat[i];
printf("\nP[%d]\t\t%d\t\t%d\t\t%d",i+1,bt[i],wt[i],tat[i]);
}
avwt/=i;
avtat/=i;
printf("\n\nAverage Waiting Time:%d",avwt);
printf("\n\nAverage Turnaround Time:%d",avtat);
return 0;
}
```

OUTPUT



```
Enter the total number of processes (maximum 20): 4

Enter Process Burst Time
P[1]: 3
P[2]: 21
P[3]: 12
P[4]: 3

Process      Burst Time    Waiting Time    Turnaround Time
P[1]         3             0              3
P[2]         21            3              24
P[3]         12            24             36
P[4]         3             36             39

Average Waiting Time: 15
Average Turnaround Time: 25

-----
Process exited after 23.29 seconds with return value 0
Press any key to continue
```

2.Shortest job first(SJF) non-preemptive process scheduling algorithm.

Introduction

In SJF under non-preemptive scheduling, once the CPU has been allocated to a process, the process keeps the CPU until it releases the CPU. the process with the smallest execution time (or burst time) is selected for execution next.

Algorithm

Step 1: Declare the array size.

Step 2: Get the number of elements to be inserted.

Step 3: Select the process which have shortest burst will execute first.

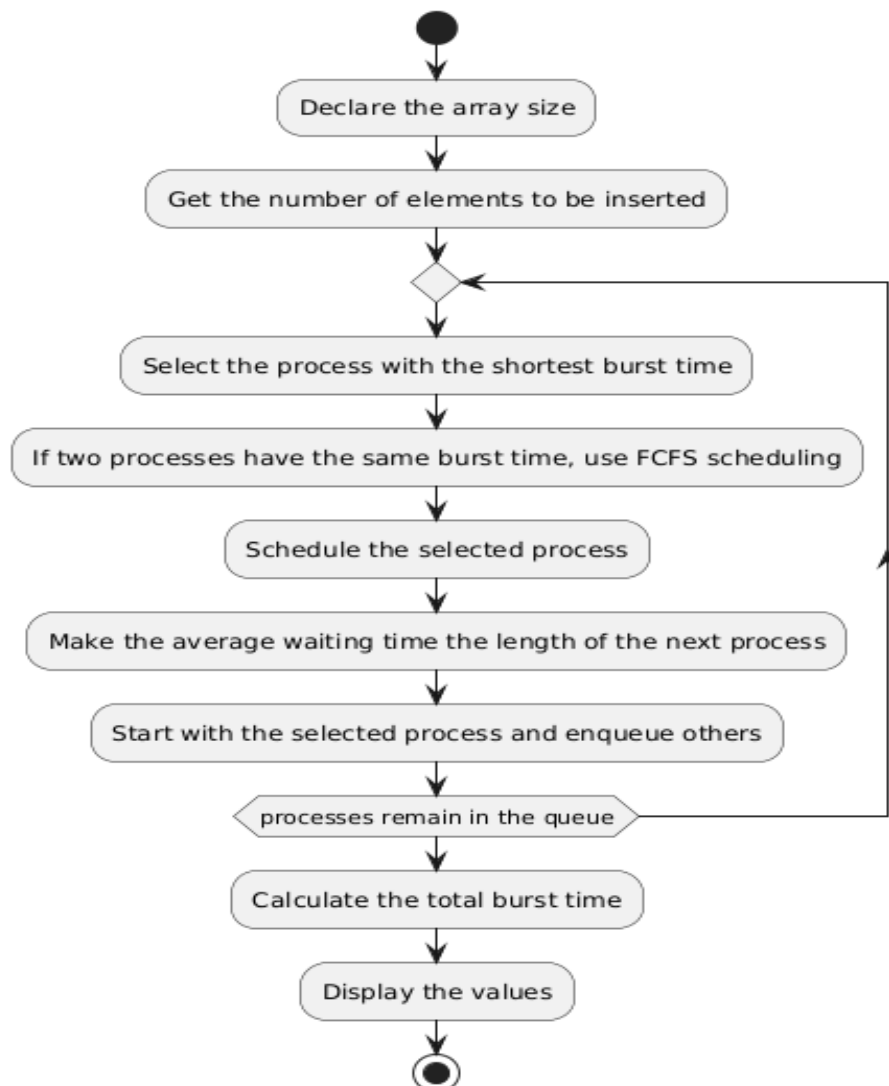
Step 4: If two process have same burst length then FCFS scheduling algorithm used.

Step 5: Make the average waiting the length of next process.

Step 6: Start with the first process from it's selection as above and let other process to be in queue.

Step 7: Calculate the total number of burst time. Step 8: Display the values.

Flowchart



Program code

```
#include<stdio.h>

int main()

{

int time, bt[10], at[10], sum_bt=0, smallest, n, i;

int sumt=0, sumw=0;

printf("no of processes : ");

scanf("%d", &n);

for(i=0; i<n; i++)

{

printf("arrival time for process P%d : ", i+1);

scanf("%d", &at[i]);

printf("burst time for process P%d : ", i+1);

scanf("%d", &bt[i]);

sum_bt+=bt[i];

}

bt[9]=9999;

for(time=0; time<sum_bt;)

{

smallest=9;

for(i=0; i<n; i++)

{

if(at[i]<=time && bt[i]>0 && bt[i]<bt[smallest])

smallest=i;

}

printf("P[%d]\t|\t%d\t|\t%d\t|\t%d\n", smallest+1, bt[smallest], time+bt[smallest]-
```

```

at[smallest],time-at[smallest]);

sumt+=time+bt[smallest]-at[smallest];

sumw+=time-at[smallest];

time+=bt[smallest];

bt[smallest]=0;

}

printf("\n\n average waiting time = %f",sumw*1.0/n);

printf("\n\n average turnaround time = %f",sumt*1.0/n);

return 0;

}

```

Output

```

Enter Total Number of Processes:      5
Enter Burst Time For Process[1]:      12
Enter Burst Time For Process[2]:      34
Enter Burst Time For Process[3]:      22
Enter Burst Time For Process[4]:      67
Enter Burst Time For Process[5]:      8

Process ID      Burst Time      Waiting Time      Turnaround Time
Process[5]      8      0      8
Process[1]      12      8      20
Process[3]      22      20      42
Process[2]      34      42      76
Process[4]      67      76      143

Average Waiting Time:  29.200001
Average Turnaround Time:  57.799999

-----
Process exited after 49.89 seconds with return value 0
Press any key to continue . . .

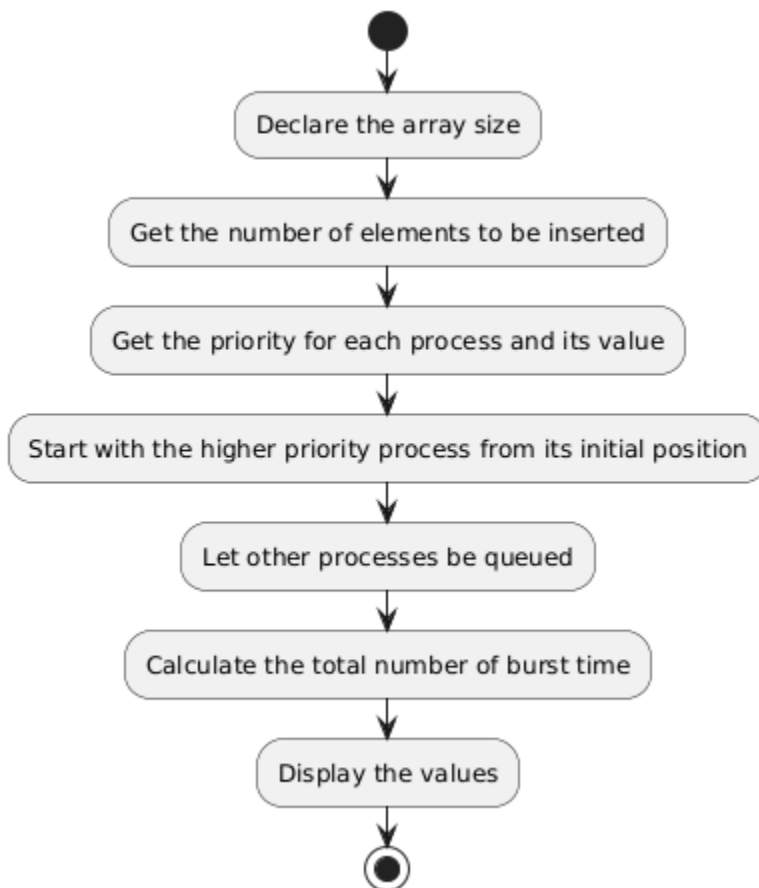
```

3. Priority based non-preemptive scheduling algorithm.

Introduction

Priority is associated with each process and the CPU allocation to the process with the highest priority when the CPU is available. In non-preemptive priority, a process will finish its execution although a process of higher priority tries to allocate the CPU when it.

Flowchart



Program Code

```
#include<stdio.h>

int main()
{
    int burst_time[20], process[20], waiting_time[20], turnaround_time[20], priority[20];
    int i, j, limit, sum = 0, position, temp;
    float average_wait_time, average_turnaround_time;
    printf("Enter Total Number of Processes:\t");
    scanf("%d", &limit);
    printf("\nEnter Burst Time and Priority For %d Processes\n", limit);
    for(i = 0; i < limit; i++)
    {
        printf("\nProcess[%d]\n", i + 1);
        printf("Process Burst Time:\t");
        scanf("%d", &burst_time[i]);
        printf("Process Priority:\t");
        scanf("%d", &priority[i]);
        process[i] = i + 1;
    }
    for(i = 0; i < limit; i++)
    {
        position = i;
        for(j = i + 1; j < limit; j++)
        {
            if(priority[j] < priority[position])
            {
                position = j;
            }
        }
        temp = priority[i];
        priority[i] = priority[position];
        priority[position] = temp;
        temp = burst_time[i];
        burst_time[i] = burst_time[position];
        burst_time[position] = temp;
        temp = process[i];
        process[i] = process[position];
        process[position] = temp;
    }
    waiting_time[0] = 0;
    for(i = 1; i < limit; i++)
    {
        waiting_time[i] = 0;
        for(j = 0; j < i; j++)
        {
            waiting_time[i] = waiting_time[i] + burst_time[j];
        }
        sum = sum + waiting_time[i];
    }
    average_wait_time = sum / limit;
    sum = 0;
```



```

printf("\nProcess ID\tBurst Time\t Waiting Time\t Turnaround Time\n");
for(i = 0; i < limit; i++)
{
    turnaround_time[i] = burst_time[i] + waiting_time[i];
    sum = sum + turnaround_time[i];
    printf("\nProcess[%d]\t\t%d\t\t %d\t\t %d\n", process[i], burst_time[i], waiting_time[i],
turnaround_time[i]);
}
average_turnaround_time = sum / limit;
printf("\nAverage Waiting Time:\t%f", average_wait_time);
printf("\nAverage Turnaround Time:\t%f\n", average_turnaround_time);

return 0;
}

```

Output

```

Enter Total Number of Processes:      3
Enter Burst Time and Priority For 3 Processes

Process[1]
Process Burst Time:    14
Process Priority:      33

Process[2]
Process Burst Time:    23
Process Priority:      43

Process[3]
Process Burst Time:    12
Process Priority:      27

Process ID      Burst Time      Waiting Time      Turnaround Time
Process[3]      12              0                12
Process[1]      14              12               26
Process[2]      23              26               49

Average Waiting Time:  12.000000
Average Turnaround Time:  29.000000

-----
Process exited after 30.22 seconds with return value 0
Press any key to continue
3/ | | | | | process[i] = process[position];
70 | | | | | -----

```

4.Priority based preemptive process scheduling algorithm.

Algorithm:

Step 1 **Input:** Get the total number of processes.

Step 2.**For each process:**

Input the **arrival time**, **burst time**, and **priority**.

Step 3.**Sort processes by arrival time** to ensure they are handled in the order they arrive.

Step 4.**Initialize variables:**

Set total burst time and process status.

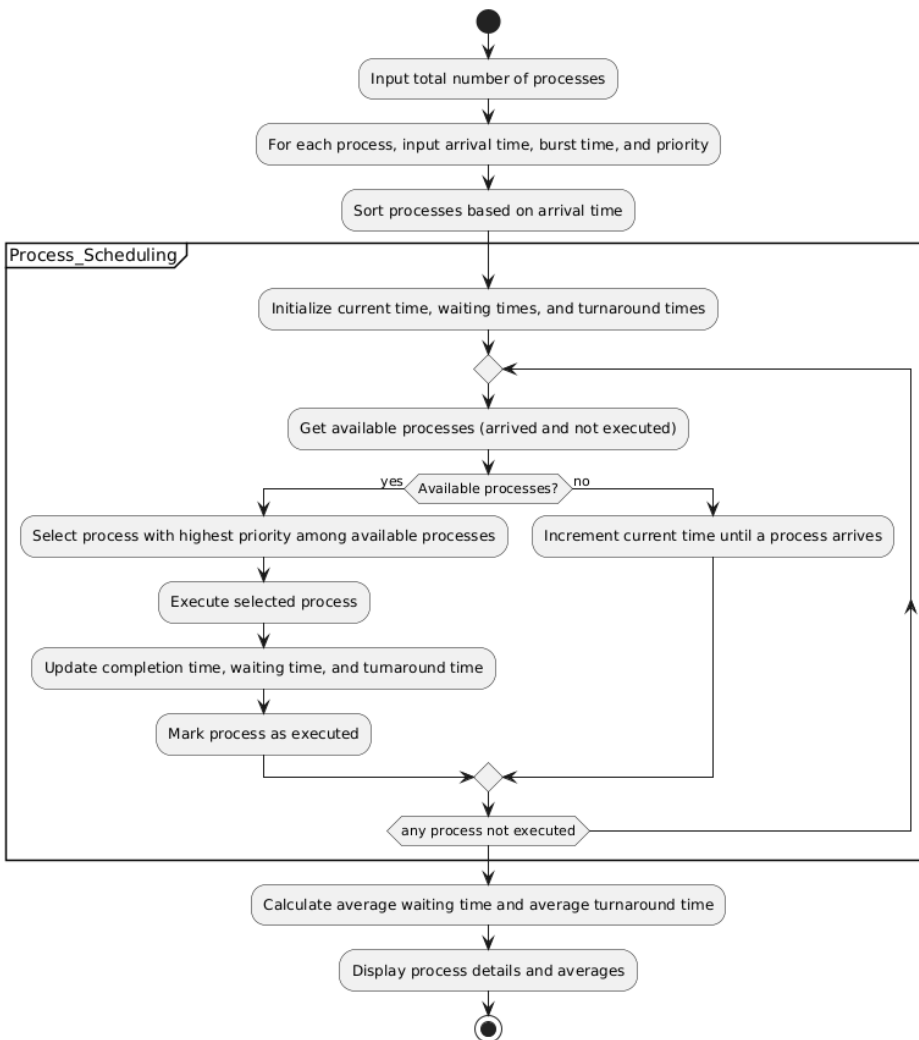
Step5. **For each time unit** (from the first process' arrival time):

- Select the process with the highest priority (among available processes).
- Calculate **completion time**, **waiting time**, and **turnaround time** for the selected process.
- Mark the process as executed.

Step 6.**Compute the average waiting time** and **average turnaround time**.

Step 7.Exit the program

Flowchart



Program code:

```
#include<stdio.h>

struct process
{
    char process_name;
    int arrival_time, burst_time, ct, waiting_time, turnaround_time, priority;
    int status;
}process_queue[10];
int limit;

void Arrival_Time_Sorting()
{
    struct process temp;
    int i, j;
    for(i = 0; i < limit - 1; i++)
    {
        for(j = i + 1; j < limit; j++)
        {
            if(process_queue[i].arrival_time > process_queue[j].arrival_time) {
                temp = process_queue[i];
                process_queue[i] = process_queue[j];
                process_queue[j] = temp; } } }
}

int main() {
    int i, time = 0, burst_time = 0, largest;
    char c;
    float wait_time = 0, turnaround_time = 0, average_waiting_time, average_turnaround_time;
    printf("\nEnter Total Number of Processes:\t");
    scanf("%d", &limit);
    for(i = 0, c = 'A'; i < limit; i++, c++)
    {
        process_queue[i].process_name = c;
        printf("\nEnter Details For Process[%C]:\n", process_queue[i].process_name);
        printf("Enter Arrival Time:\t");
        scanf("%d", &process_queue[i].arrival_time );
        printf("Enter Burst Time:\t");
        scanf("%d", &process_queue[i].burst_time);
        printf("Enter Priority:\t");
        scanf("%d", &process_queue[i].priority);
        process_queue[i].status = 0;
        burst_time = burst_time + process_queue[i].burst_time;
    }
    Arrival_Time_Sorting();
    process_queue[9].priority = -9999;
    printf("\nProcess Name\tArrival Time\tBurst Time\tPriority\tWaiting Time");
    for(time = process_queue[0].arrival_time; time < burst_time;) {
        largest = 9;
        for(i = 0; i < limit; i++)
        {
            if(process_queue[i].arrival_time <= time && process_queue[i].status != 1 &&
process_queue[i].priority > process_queue[largest].priority)
```

```

        {
            largest = i; } }
    time = time + process_queue[largest].burst_time;
    process_queue[largest].ct = time;
    process_queue[largest].waiting_time = process_queue[largest].ct - process_queue[largest].arrival_time
- process_queue[largest].burst_time;
    process_queue[largest].turnaround_time = process_queue[largest].ct -
process_queue[largest].arrival_time;
    process_queue[largest].status = 1;
    wait_time = wait_time + process_queue[largest].waiting_time;
    turnaround_time = turnaround_time + process_queue[largest].turnaround_time;
    printf("\n%c\t\t%d\t\t%d\t\t%d\t\t%d", process_queue[largest].process_name,
process_queue[largest].arrival_time, process_queue[largest].burst_time, process_queue[largest].priority,
process_queue[largest].waiting_time);
}
    average_waiting_time = wait_time / limit;
    average_turnaround_time = turnaround_time / limit;
    printf("\n\nAverage waiting time:\t%f\n", average_waiting_time);
    printf("Average Turnaround Time:\t%f\n", average_turnaround_time);
}

```

Output

```

Process Priority:      4

Process[2]
Process Burst Time:   12
Process Priority:      3

Process[3]
Process Burst Time:   55
Process Priority:      1

Process[4]
Process Burst Time:   34
Process Priority:      23

Process ID            Burst Time      Waiting Time      Turnaround Time
Process[3]             55              0                 55
Process[2]             12              55                 67
Process[1]             23              67                 90
Process[4]             34              90                124

Average Waiting Time:  53.000000
Average Turnaround Time: 84.000000

```

5.Round Robin process scheduling with arrival time(preemptive scheduling algorithm)

Introduction

Round Robin (RR) process scheduling algorithm treat with the ready queue as a FIFO queue as processes. Processes are added to the tail of the ready queue.

In the round robin algorithm, every process gets an equal time of execution which is defined by the quantum time. Therefore, no process will be able to hold the CPU for a longer time period.

The round-robin job scheduling algorithm is, therefore, used in a multi-user, timesharing or multi-tasking operating systems.

Program Code

```
#include<stdio.h>
int main() {
int count,j,n,time,remain,flag=0,time_quantum;
int wait_time=0,turnaround_time=0,at[10],bt[10],rt[10];
printf("Enter Total Process:\t "); scanf("%d",&n);
remain=n;
for(count=0;count<n;count++)
{
printf("Enter Arrival Time and Burst Time for Process Process Number %d
:",count+1);
scanf("%d",&at[count]);
scanf("%d",&bt[count]);
rt[count]=bt[count];
}
printf("Enter Time Quantum:\t");
scanf("%d",&time_quantum);
printf("\n\nProcess\t|Turnaround Time|Waiting Time\n\n");
for(time=0,count=0;remain!=0;)
{
if(rt[count]<=time_quantum && rt[count]>0)
{
time+=rt[count];
rt[count]=0;
flag=1;
}
else if(rt[count]>0)
{
rt[count]-=time_quantum;
time+=time_quantum;
}
if(rt[count]==0 && flag==1)
{
remain--;
printf("P[%d]\t|\t%d\t|\t%d\n",count+1,time-at[count],time-at[count]-bt[count]);
wait_time+=time-at[count]-bt[count];
turnaround_time+=time-at[count];
flag=0;
}
```

```

}
if(count==n-1)
count=0;
else if(at[count+1]<=time)
count++;
else
count=0; }
printf("\nAverage Waiting Time= %f\n",wait_time*1.0/n);
printf("Avg Turnaround Time = %f",turnaround_time*1.0/n);
return 0;
}

```

Output

```

] Enter Total Number of Processes:      3
Enter Details of Process[1]
Arrival Time:  2
Burst Time:    3

Enter Details of Process[2]
Arrival Time:  0
Burst Time:    5

Enter Details of Process[3]
Arrival Time:  3
Burst Time:    2

Enter Time Quantum:      2

Process ID          Burst Time      Turnaround Time      Waiting Time
Process[3]          2              3                  1
Process[1]          3              5                  2
Process[2]          5              10                 5

Average Waiting Time:  2.666667
Avg Turnaround Time:  6.000000

-----
Process exited after 92.43 seconds with return value 0

```

Lab-5

Program to simulate page replacement algorithm :FIFO, LRU, LRU.

Introduction

A page replacement algorithm is a method used in operating systems to manage the contents of the page table and decide which memory pages to swap out when a page fault occurs. When a program accesses a page that is not currently in physical memory (RAM), the operating system must bring that page into memory, which may require evicting an existing page to make space.

1.Least recently used (LRU)

This algorithm replaces the page that has not been used for the longest period of time. It is based on the assumption that pages used recently will likely be used again soon.

Algorithm

Step1.Read the number of page references (m).

Step 2.Read the reference string into an array rs[].

Step 3.Read the number of available frames (f).

Step 4.Initialize:

Create an array a[] to hold the current pages in frames (initialized to -1).

Create an array cntr[] to count the usage frequency of each frame (initialized to 0).

Set pf (page fault count) to 0.

Step 5.Page Replacement Process:

For each page reference rs[i]:

Check if rs[i] is already in a[]:

If found, increment the corresponding counter in cntr[].

If not found:

Find the frame with the least frequency (minimum in cntr[]).

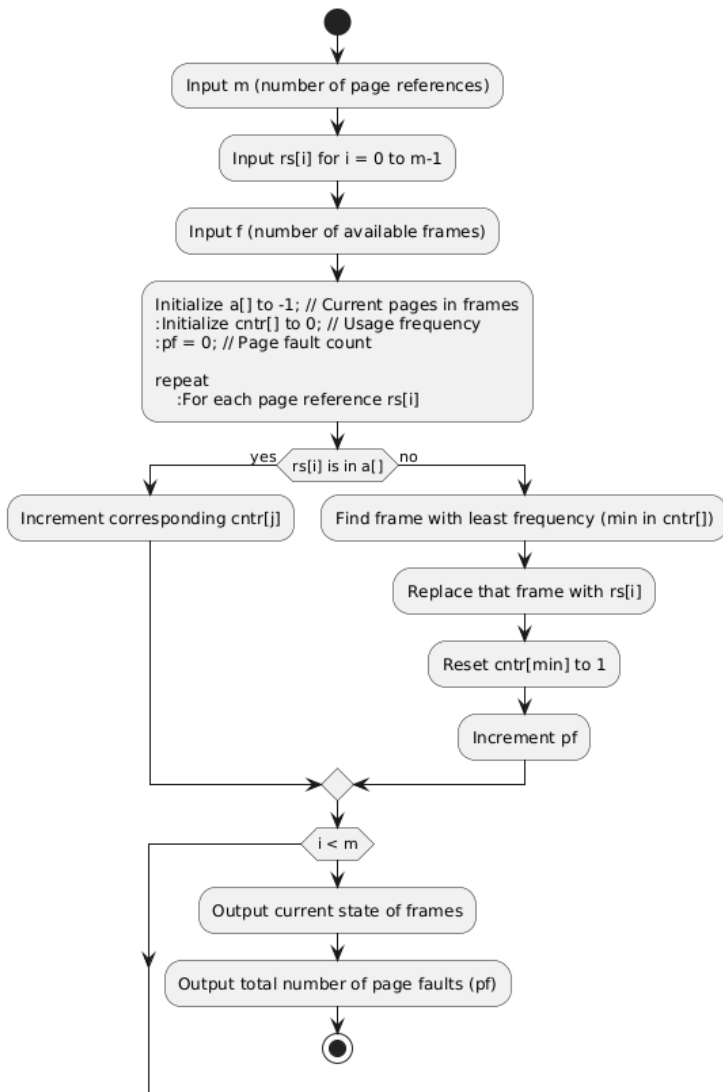
Replace that frame with rs[i].

Reset the counter for the new page and increment pf.

Step 6. Print the current state of frames after each reference.

Step 7. Print the total number of page faults at the end.

Flowchart



Program code

```
#include <stdio.h>
#define MAX_REFERENCES 50
#define MAX_FRAMES 20
int main() {
    int rs[MAX_REFERENCES];
    int i, j, k, m, f;
    int cntr[MAX_FRAMES];
    int a[MAX_FRAMES];
    int min, pf = 0;
    printf("\nEnter number of page references: ");
    scanf("%d", &m);
    printf("\nEnter the reference string: ");
    for (i = 0; i < m; i++) {
        scanf("%d", &rs[i]); }
    printf("\nEnter the available number of frames: ");
    scanf("%d", &f);
    for (i = 0; i < f; i++) {
        cntr[i] = 0;
        a[i] = -1;
    }
    printf("\nThe Page Replacement Process is:\n");
    for (i = 0; i < m; i++) {
        for (j = 0; j < f; j++) {
            if (rs[i] == a[j]) {
                cntr[j]++;
                break;    } }
        if (j == f) {
            min = 0;
            for (k = 1; k < f; k++) {
                if (cntr[k] < cntr[min]) {
                    min = k; } }
            a[min] = rs[i];
            cntr[min] = 1;
            pf++;
        }
        for (j = 0; j < f; j++) {
            printf("\t%d", a[j]); }
        if (j == f) {
            printf("\tPF No. %d", pf); }
        printf("\n"); }
    printf("\nTotal number of page faults: %d\n", pf);
    return 0;}
```

Output

```
Enter the reference string: 2
1
2
3
4
5
6
6
3
2

Enter the available number of frames: 3

The Page Replacement Process is:
  2    -1    -1    PF No. 1
  2     1    -1    PF No. 2
  2     1    -1    PF No. 2
  2     1     3    PF No. 3
  2     4     3    PF No. 4
  2     5     3    PF No. 5
  2     6     3    PF No. 6
  2     6     3    PF No. 6
  2     6     3    PF No. 6
  2     6     3    PF No. 6

Total number of page faults: 6
-----
```

First-In First-Out

The oldest page in memory (the one that has been in memory the longest) is replaced. This algorithm is simple to implement but can lead to suboptimal performance, known as the "Belady's anomaly," where increasing the number of page frames can lead to an increase in page faults.

Algorithm

Step 1. Read the length of the reference string (n).

Step 2. Read the reference string into an array rs[].

Step 3. Read the number of available frames (f).

Step 4. Initialize:

Create an array m[] to hold the current pages in frames (initialized to -1).

Set pf (page fault count) to 0 and count to 0.

Step 5. Process Each Page Reference:

For each page rs[i]:

If rs[i] is not in m[]:

Add rs[i] to m[count].

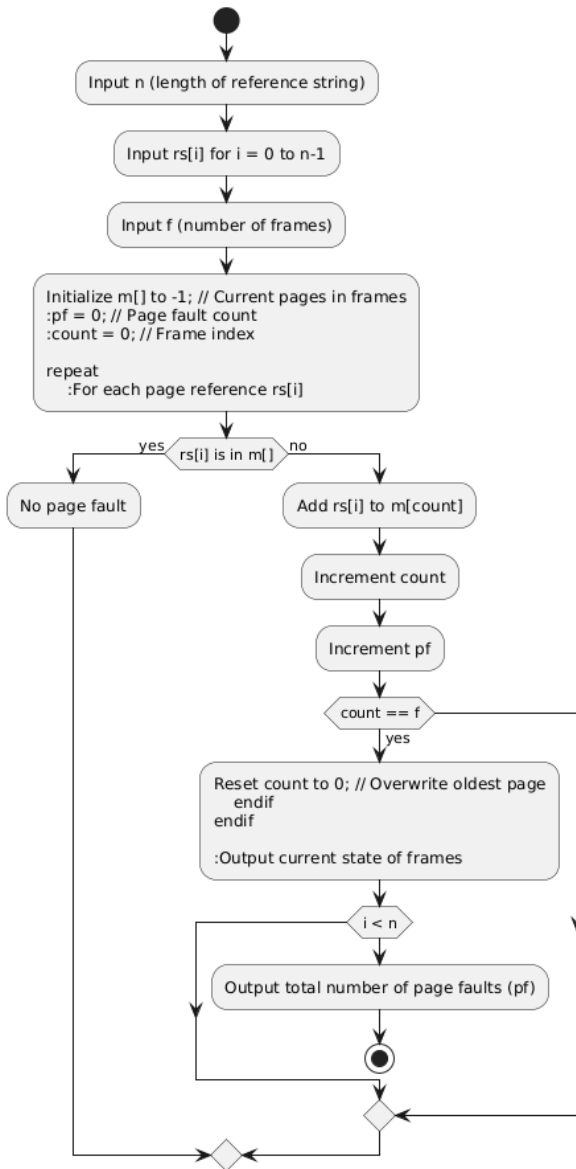
Increment count and pf.

If count equals f, reset count to 0 (to overwrite the oldest page).

Print the current state of frames.

Step 6. Print the total number of page faults (pf).

Flowchart



Program Code

```
#include <stdio.h>
#define MAX_FRAMES 10
#define MAX_REFERENCES 25
int main() {
    int i, j, k, f, pf = 0, count = 0;
    int rs[MAX_REFERENCES];
    int m[MAX_FRAMES];
    int n;
    printf("\nEnter the length of the reference string: ");
    scanf("%d", &n);
    printf("\nEnter the reference string: ");
    for (i = 0; i < n; i++) {
        scanf("%d", &rs[i]);
    }
    printf("\nEnter the number of frames: ");
    scanf("%d", &f);
    for (i = 0; i < f; i++) {
        m[i] = -1;
    }
    printf("\nThe Page Replacement Process is:\n");
    for (i = 0; i < n; i++) {
        for (k = 0; k < f; k++) {
            if (m[k] == rs[i]) {
                break; } }
        if (k == f) {
            m[count] = rs[i];
            count++;
            pf++; }
        for (j = 0; j < f; j++) {
            printf("\t%d", m[j]); }
        if (k == f) {
            printf("\tPF No. %d", pf); }
        printf("\n");
        if (count == f) {
            count = 0; } }
    printf("\nThe number of Page Faults using FIFO are: %d\n", pf);
    return 0;
}
```

Output

```
Enter the length of the reference string: 10
Enter the reference string: 2 1 2 3 4 5 3 6 7 8
Enter the number of frames: 3
The Page Replacement Process is:
  2    -1    -1    PF No. 1
  2     1    -1    PF No. 2
  2     1    -1
  2     1     3    PF No. 3
  4     1     3    PF No. 4
  4     5     3    PF No. 5
  4     5     3
  4     5     6    PF No. 6
  7     5     6    PF No. 7
  7     8     6    PF No. 8

The number of Page Faults using FIFO are: 8
-----
Process exited after 40.38 seconds with return value 0
```

LFU(Least frequently used)

This algorithm replaces the page that has been used the least frequently. It keeps track of how often each page is accessed and evicts the one with the lowest access count.

Algorithm

Step1.Read the number of page references (m).

Step 2.Read the reference string into an array rs[].

Step 3.Read the number of available frames (f).

Step 4.Initialize:

Create an array a[] to hold the current pages in frames (initialized to -1).

Create an array cntr[] to count the usage frequency of each frame (initialized to 0).Set pf (page fault count) to 0.

Step 5.Process Each Page Reference:

For each page reference rs[i]:

Check if rs[i] is already in a[]:

If found, increment the corresponding counter in cntr[].

If not found:

Find the frame with the least frequency (minimum in cntr[]).

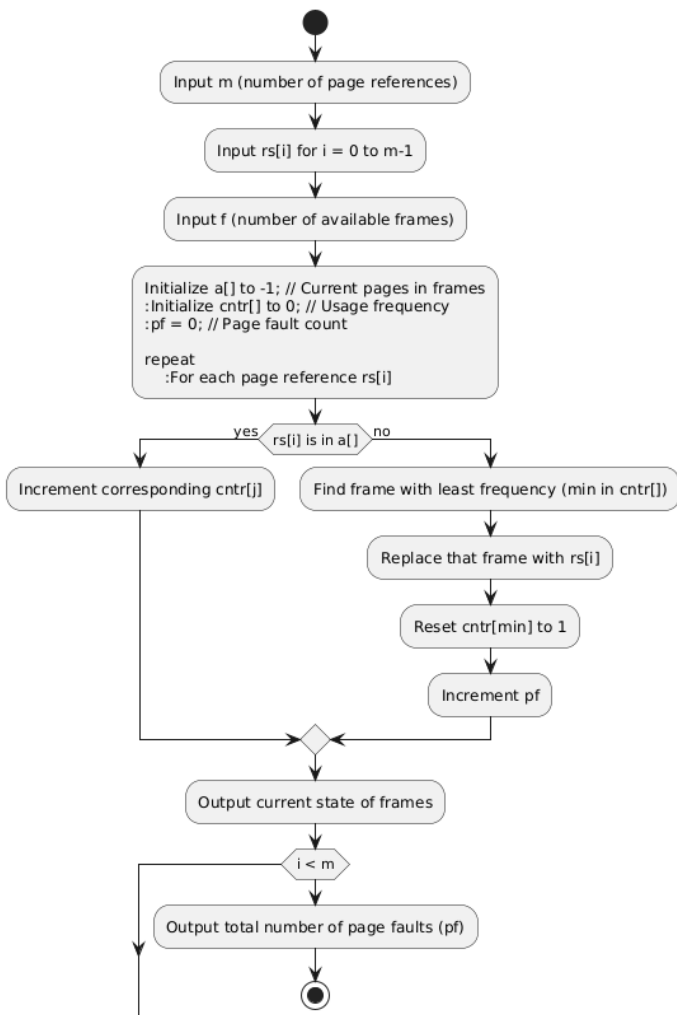
Replace that frame with rs[i].

Reset the counter for the new page and increment pf.

Step 6.Print the current state of frames after each reference.

Step 7.Print the total number of page faults at the end.

Flowchart



Program Code

```
#include <stdio.h>
#define MAX_REFERENCES 50
#define MAX_FRAMES 20
int main() {
    int rs[MAX_REFERENCES];
    int i, j, k, m, f;
    int cntr[MAX_FRAMES];
    int a[MAX_FRAMES];
    int min, pf = 0;
    printf("\nEnter number of page references: ");
    scanf("%d", &m);
    printf("\nEnter the reference string: ");
    for (i = 0; i < m; i++) {
        scanf("%d", &rs[i]);
    }
    printf("\nEnter the available number of frames: ");
    scanf("%d", &f);
    for (i = 0; i < f; i++) {
        cntr[i] = 0;
        a[i] = -1;
    }
    printf("\nThe Page Replacement Process is:\n");
    for (i = 0; i < m; i++) {
        for (j = 0; j < f; j++) {
            if (rs[i] == a[j]) {
                cntr[j]++;
                break;
            }
        }
        if (j == f) {
            min = 0;
            for (k = 1; k < f; k++) {
                if (cntr[k] < cntr[min]) {
                    min = k;
                }
            }
            a[min] = rs[i];
            cntr[min] = 1;
            pf++;
        }
        for (j = 0; j < f; j++) {
            printf("\t%d", a[j]);
        }
        if (j == f) {
            printf("\tPF No. %d", pf);
        }
        printf("\n");
    }
    printf("\nTotal number of page faults: %d\n", pf);
    return 0;
}
```


Output

```
Enter number of page references: 12
Enter the reference string: 2 3 6 4 3 9 6 1 5 7 3 2
Enter the available number of frames: 3
The Page Replacement Process is:
    2      -1      -1      PF No. 1
    2       3      -1      PF No. 2
    2       3       6      PF No. 3
    4       3       6      PF No. 4
    4       3       6      PF No. 4
    9       3       6      PF No. 5
    9       3       6      PF No. 5
    1       3       6      PF No. 6
    5       3       6      PF No. 7
    7       3       6      PF No. 8
    7       3       6      PF No. 8
    2       3       6      PF No. 9

Total number of page faults: 9

-----
Process exited after 51.91 seconds with return value 0
Press any key to continue . . .
```

LAB -6

Program to simulate Disk Scheduling algorithm: FCFS, SCAN

1.FIRST-COME FIRST - SERVE

Algorithm

Step1.Read the number of processes (total_process).

Step 2Read the burst times for each process into an array burst_time[].

Step 3.Initialize:

Set waiting_time[0] to 0 (the first process has no waiting time).

Step 4.Calculate Waiting Time:

For each process from 1 to total_process - 1:

Calculate waiting_time[i] as the sum of burst_time[0] to burst_time[i-1].

Step 5.Calculate Turnaround Time:

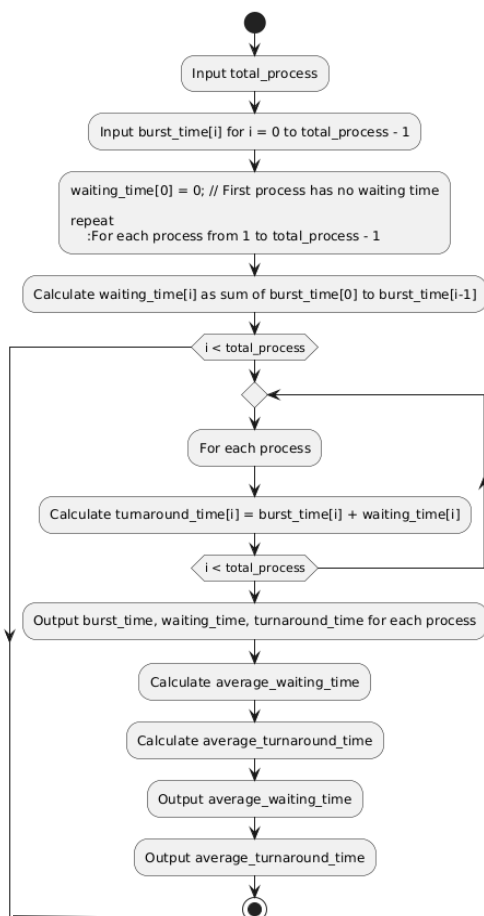
For each process:

Calculate turnaround_time[i] as burst_time[i] + waiting_time[i].

Step 6.Print the burst time, waiting time, and turnaround time for each process.

Step 7.Calculate and print the average waiting time and average turnaround time.

Flowchart



Program code

```
#include<stdio.h>

int main()
{
    float burst_time[30], waiting_time[30], turnaround_time[30];
    float average_waiting_time = 0.0, average_turnaround_time = 0.0;
    int count, j, total_process;
    printf("Enter The Number of Processes To Execute:\t");
    scanf("%d", &total_process);
    printf("\nEnter The Burst Time of Processes:\n\n");
    for(count = 0; count < total_process; count++)
    {
        printf("Process [%d]:", count + 1);
        scanf("%f", &burst_time[count]);
    }
    waiting_time[0] = 0;
    for(count = 1; count < total_process; count++)
    {
        waiting_time[count] = 0;
        for(j = 0; j < count; j++)
        {
            waiting_time[count] = waiting_time[count] + burst_time[j];
        }
    }
    printf("\nProcess\t\tBurst Time\tWaiting Time\tTurnaround Time\n");
    for(count = 0; count < total_process; count++)
    {
        turnaround_time[count] = burst_time[count] + waiting_time[count];
        average_waiting_time = average_waiting_time + waiting_time[count];
        average_turnaround_time = average_turnaround_time + turnaround_time[count];
        printf("\nProcess [%d]\t\t%.2f\t\t%.2f\t\t%.2f", count + 1, burst_time[count], waiting_time[count],
turnaround_time[count]);
    }
    printf("\n");
    average_waiting_time = average_waiting_time / count;
    average_turnaround_time = average_turnaround_time / count;
    printf("\nAverage Waiting Time = %f", average_waiting_time);
    printf("\nAverage Turnaround Time = %f", average_turnaround_time);
    printf("\n");
    return 0;
}
```

Output

```
Enter The Number of Processes To Execute:      4

Enter The Burst Time of Processes:

Process [1]:10
Process [2]:15
Process [3]:20
Process [4]:30

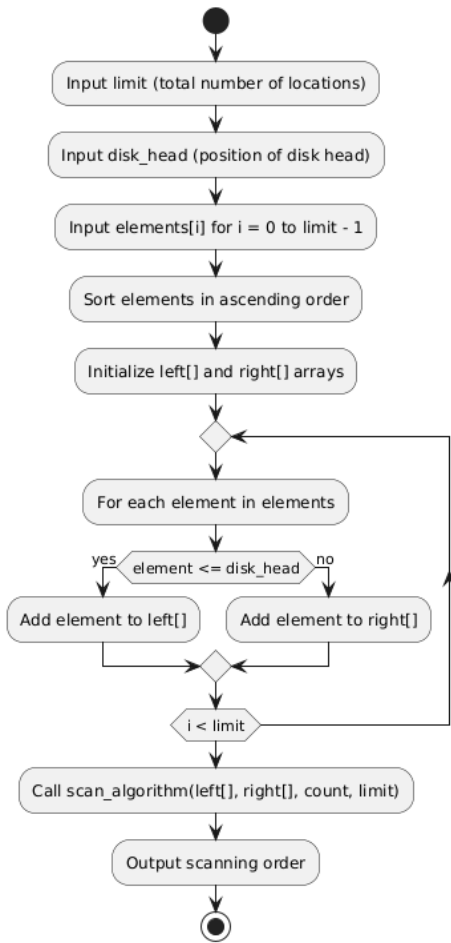
Process      Burst Time      Waiting Time      Turnaround Time
Process [1]      10.00      0.00      10.00
Process [2]      15.00      10.00      25.00
Process [3]      20.00      25.00      45.00
Process [4]      30.00      45.00      75.00

Average Waiting Time = 20.000000
Average Turnaround Time = 38.750000
```

2.SCAN disk scheduling algorithm

Algorithm

1. **Input:**
 - Read the total number of disk locations (**limit**).
 - Read the position of the disk head (**disk_head**).
 - Read the disk request queue into an array **elements[]**.
2. **Sort Requests:**
 - Sort the disk requests in ascending order.
3. **Divide Requests:**
 - Initialize two arrays: **left[]** for requests less than or equal to **disk_head** and **right[]** for requests greater than **disk_head**.
 - Iterate through the sorted requests:
 - If a request is less than or equal to **disk_head**, add it to **left[]**.
 - Otherwise, add it to **right[]**.
4. **Scan Algorithm:**
 - Create a combined order of servicing requests:
 - First, service the **left[]** requests in reverse order.
 - Then, service the **disk_head** position.
 - Finally, service the **right[]** requests in order.
5. **Output:**
 - Print the order in which the disk requests will be serviced.



Program code

```

#include <stdio.h>
#define MAX_ELEMENTS 20
void scan_algorithm(int left[], int right[], int count, int limit) {
    int arr[MAX_ELEMENTS];
    int x = count - 1, y = count, c = 0, d = 0;
    while (x >= 0) {
        arr[d++] = left[x--];
    }
    arr[d++] = 0;
    while (y < limit) {
        arr[d++] = right[c++];
    }
    printf("\nScanning Order:\n");
    for (int j = 0; j < d; j++) {
        printf("%d ", arr[j]);
    }
    printf("\n");
}
void division(int elements[], int limit, int disk_head) {

```

```

int count = 0, p = 0, q = 0;
int left[MAX_ELEMENTS], right[MAX_ELEMENTS];
while (count < limit && elements[count] <= disk_head) {
    count++; }
for (q = 0; q < count; q++) {
    left[q] = elements[q]; }
for (p = 0; p < limit - count; p++) {
    right[p] = elements[count + p];}
scan_algorithm(left, right, count, limit);
}
void sorting(int elements[], int limit) {
    for (int count = 0; count < limit - 1; count++) {
        int small = elements[count];
        int location = count;
        for (int j = count + 1; j < limit; j++) {
            if (small > elements[j]) {
                small = elements[j];
                location = j; } }
        int temp = elements[location];
        elements[location] = elements[count];
        elements[count] = temp; } }
int main() {
    int elements[MAX_ELEMENTS], limit, disk_head;
    printf("Enter total number of locations: ");
    scanf("%d", &limit);
    printf("Enter position of disk head: ");
    scanf("%d", &disk_head);
    printf("Enter elements of disk head queue:\n");
    for (int count = 0; count < limit; count++) {
        printf("Element[%d]: ", count + 1);
        scanf("%d", &elements[count]);}
    sorting(elements, limit);
    division(elements, limit, disk_head);
    return 0; }

```

Output

```
Enter total number of locations: 3
Enter position of disk head: 3
Enter elements of disk head queue:
Element[1]: 1
Element[2]: 2
Element[3]: 3

Scanning Order:
3 2 1 0

-----
Process exited after 26.28 seconds with return value 0
Press any key to continue . . .
```